EduOS CPU Scheduling Simulator - Detailed Final Report

This project is a CPU scheduling simulator built for Operating Systems coursework. It demonstrates how an operating system manages processes using different scheduling algorithms.

1. Introduction: In a real operating system, multiple processes compete for CPU time. CPU scheduling decides which process runs first to improve efficiency and fairness.

2. Project Goal: The goal is to simulate CPU scheduling algorithms and compare their performance using metrics like waiting time and turnaround time.

3. Process Structure: Each process has a Process ID (PID), arrival time (when it enters system), burst time (CPU time needed), and priority (importance level).

4. FCFS (First Come First Serve): This is the simplest algorithm. Processes are executed in the order they arrive. It is easy but can cause long waiting times.

5. SJF (Shortest Job First): This algorithm selects the process with the smallest burst time first. It reduces average waiting time but may cause starvation.

6. Round Robin (RR): Each process gets a fixed time slice (quantum). If not finished, it goes back to the queue. This ensures fairness.

7. Priority Scheduling: Processes are executed based on priority value. Higher priority processes run first.

8. System Design: The system is divided into Python scheduler (algorithms), controller (main execution), and metrics module (performance comparison).

9. Working Flow: Processes are generated randomly, passed to scheduling algorithms, executed step by step, and results are collected.

10. Metrics: Waiting time is time spent waiting in queue. Turnaround time is total time from arrival to completion.

11. Testing: The system was tested using multiple random process sets to ensure correctness of scheduling logic.

12. Conclusion: The project successfully demonstrates how CPU scheduling works in operating systems and how different algorithms affect performance.